

Performance Analysis of Parallel Morphological Image Opening using OpenMP

Abstract

This report explores the parallelization of the morphological Opening operation (Erosion followed by Dilation) using OpenMP in C+. The objective was to reduce processing times for two distinct workloads: high-resolution medical images of the eye and simpler satellite images of water bodies.

Approach summary: We implemented a data-parallel strategy using the `#pragma omp parallel for collapse(2)` directive for 9×9 kernel operations. The performance of static and dynamic scheduling was compared on an Intel i7-8550U processor (4 physical cores, 8 logical threads).

Key results show that the optimal configuration was reached at 8 threads, matching the system's logical core limit. We achieved a maximum speedup of 4.14x for the high-complexity dataset and 7.25x for the simpler images. Beyond 8 threads, efficiency dropped significantly due to hardware limitations and the increasing cost of managing additional threads.

1. Introduction

1.1. Problem Description and Motivation

Morphological image processing is a fundamental tool in computer vision, particularly for noise removal and boundary smoothing. This project focuses on the Opening operation, which consists of an Erosion followed by a Dilation. This filter is essential for cleaning medical images, such as those used for detecting diabetic retinopathy, and for smoothing the contours of geographical features like water bodies in satellite imagery. The primary motivation is to reduce the high execution times required to process these large and complex datasets.

1.2. Computational Challenges

The main challenge lies in the computational intensity of the operation. For a 9×9 kernel, the algorithm must visit 81 neighboring pixels for every single pixel in the image to find the minimum (Erosion) or maximum (Dilation) value. When dealing with high-resolution images, this translates to billions of operations. Furthermore, managing the parallelization overhead, such as thread creation and synchronization, is difficult when the workload is small or when the hardware limits are reached.

1.3. Objectives and Scope

The main goals of this project are:

- Implement both sequential and parallel versions of the Opening algorithm in C++ using the OpenMP framework.
- Scheduling Analysis: Evaluate and compare the impact of static and dynamic scheduling strategies on overall performance and thread management overhead.
- Evaluate performance by measuring execution time, Speedup, and Parallel Efficiency across different thread counts.
- Analyze the scalability of the implementation using two distinct datasets: high-volume retinopathy images and low-volume satellite images.
- Identify the optimal thread count and scheduling strategy for an Intel i7-8550U processor (4 physical cores, 8 logical threads).

2. Algorithm description

2.1. Sequential Algorithm Explanation

The morphological Opening operation is a two-step spatial filter used for noise reduction and boundary smoothing. It consists of an Erosion followed by a Dilation using a 9x9 square kernel.

- Erosion: The algorithm scans the image and replaces each pixel's value with the minimum intensity found within the 9x9 neighborhood. This effectively eliminates small bright noise and shrinks foreground objects.
- Dilation: The process is then reversed using the eroded image as input. Each pixel is replaced by the maximum value in its neighborhood. This restores the size of the objects that survived the erosion while keeping the image clean. Both steps are implemented using nested loops: two outer loops to navigate the image coordinates and two inner loops to process the 81 pixels of the 9x9 kernel.

2.2. Identification of Parallelizable Sections

This algorithm is highly suitable for parallelization because it follows a data-parallel model where each pixel's output is independent of its neighbors within the same phase. The primary sections identified for parallelization are the **two outer loops** (height and width) of both the Erosion and Dilation functions. Since there are no data dependencies between pixels during a single pass, these loops can be distributed across multiple threads. The only synchronization requirement is a barrier between the Erosion and Dilation stages, ensuring the first operation is completely finished before the second one starts.

3. Parallel Implementation

3.1. Parallelization Strategy

The parallelization of the Opening operation follows a data-parallel strategy. Since each pixel in the output image can be computed independently during the Erosion or Dilation phases,

the workload is divided among multiple threads by partitioning the image pixels. To maximize the available workload and improve load balancing, we flattened the two outer loops (representing the height and width of the image) into a single iteration space.

3.2. OpenMP Directives Used

The core of the implementation relies on the following OpenMP directive applied to the nested loops of both the Erosion and Dilation functions:

```
#pragma omp parallel for collapse(2) schedule(static).
```

- **collapse(2):** This directive merges the two outermost loops, providing a larger iteration space for the threads to work on.
- **schedule(static):** We primarily used static scheduling because it offers low overhead for uniform computational tasks like image kernels. We also tested dynamic scheduling to evaluate its performance against the static approach.

3.3. Synchronization Mechanisms

Because the Opening operation is defined as an Erosion followed by a Dilation, a synchronization point is necessary. The Dilation stage cannot begin until every pixel of the image has been processed by the Erosion stage. OpenMP provides an implicit barrier at the end of the `#pragma omp parallel for` directive, ensuring that all threads complete the Erosion phase before any thread starts the Dilation phase.

3.4. Data Structures and Memory Layout

The implementation uses separate buffers for the input image, the intermediate eroded result, and the final dilated output. This avoids race conditions where a thread might read a pixel value that has already been modified by another thread. The memory layout is a flat array or a 2D matrix representing the pixel intensities, which allows for contiguous memory access.

3.5. Challenges Encountered and Solutions

- **Parallelization Overhead:** In smaller workloads (Dataset 2), the time spent creating and managing threads can consume a significant portion of the total execution time. To address this, we used static scheduling to keep overhead as low as possible.
- **Diminishing Returns:** We observed that efficiency drops as the number of threads increases, especially beyond the 4 physical cores of the i7-8550U. This is caused by Hyper-Threading limits and the fixed cost of parallel overhead.
- **Hardware Bottlenecks:** Testing with 16 or 32 threads resulted in lower performance compared to 8 threads, highlighting the impact of reaching the logical processor limit.

4. Experimental Setup

4.1. Hardware Specifications

The performance benchmarks were conducted on a laptop equipped with an Intel Core i7-8550U processor. The detailed specifications are as follows:

- **CPU:** Intel(R) Core(TM) i7-8550U running at 1.80GHz.
- **Cores:** 4 physical cores and 8 logical processors utilizing Hyper-Threading technology.
- **RAM:** 16.0 GB DDR4.

4.2. Software Environment

The application was developed in C++ and utilized the OpenMP framework for multi-threaded execution. The specific environment used for development and testing was:

- **Operating System:** Windows 11.
- **IDE:** Visual Studio Code (VS Code).
- **Compiler:** g++ 15.2.0 (MinGW-W64).
- **Compilation Flags:**
 - -fopenmp: Enables OpenMP multi-threading support.
 - -O2: Applies level 2 optimizations for a balance between performance and compilation speed.
 - -std=c++17: Ensures compatibility with the C++17 standard features.

4.3. Datasets Used

Two distinct datasets with different computational requirements were used to evaluate the implementation:

- **Test Set 1 (High-Volume):** Images from the *Indian Diabetic Retinopathy Image Dataset*. These are high-resolution images with complex structures, resulting in a large processing workload.
- **Test Set 2 (Low-Volume):** *Satellite Images of Water Bodies*. These images have smaller sizes and lower overall processing requirements, making them more sensitive to parallelization overhead costs.

4.4. Parameters Tested

The experiments focused on measuring how the algorithm scales under different configurations:

- **Thread Counts:** Tests were performed using 1, 2, 4, 8, 16, and 32 threads.
- **Scheduling Strategies:** A comparison between Static and Dynamic scheduling was conducted to analyze load balancing efficiency and overhead.
- **Kernel Size:** A constant **9x9** square structural element was used for all Erosion and Dilation operations.

4.5. Measurement Methodology

To ensure accurate results, we used the following approach:

- **Timing:** Execution times were measured using `omp_get_wtime()` to capture high-resolution wall-clock time.
- **Data Collection:** Results were automatically logged into CSV files, recording thread counts (1 to 32), scheduling types, execution times, Speedup, and Efficiency.
- **Validation:** Every parallel run was verified by comparing its output pixel-by-pixel against the sequential baseline to ensure correctness, marked as "OK" in the final tables.
- **Performance metrics:** sequential time (T_s), parallel time (T_p), speed up ($S = T_s / T_p$), efficiency ($E = S / P$) where P represents the number of threads.

5. Results and Analysis

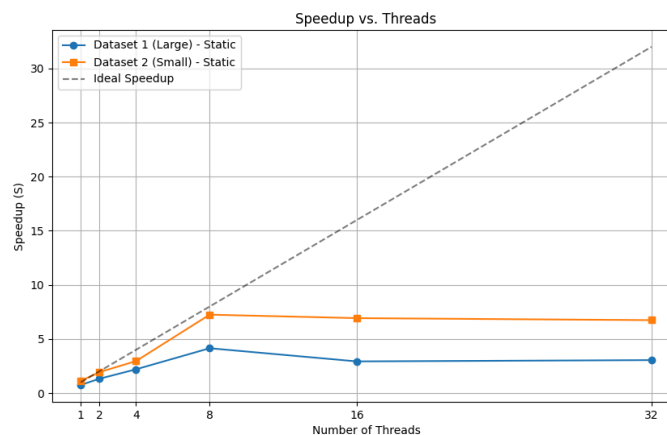
5.1. Correctness Validation

To ensure that parallelization did not compromise the accuracy of the morphological operations, we performed a pixel-by-pixel comparison between the output of the sequential algorithm and the parallel version for every test case. All results included in this analysis were verified as identical to the baseline (marked as "OK" in the datasets).

5.2. Performance Results and Speedup Curves

The performance was evaluated using two distinct workloads. The results show a clear trend: speedup increases until the logical thread limit (8 for hyper threading) is reached, after which performance degrades as you can see on the *plot1*.

- **Dataset 1 (High Complexity - Eye):** The maximum speedup achieved was **4.14x** using 8 threads with static scheduling.
- **Dataset 2 (Low Complexity - Water):** This dataset showed superior scaling, reaching a maximum speedup of **7.25x** with 8 threads.

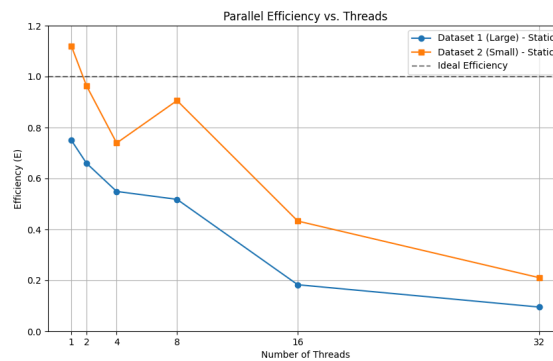


Plot1: Speedup vs threads (Dataset 1 and Dataset 2)

5.3. Scalability Analysis

This project follows a Strong Scaling model, where the problem size remains constant as we increase the number of processors.

- **Efficiency:** For both datasets, efficiency is highest at 2 and 4 threads (near the physical core limit) and drops significantly at 8 threads due to the limitations of Hyper-Threading. As you can see on the *plot2*
- **Hyper-Threading Impact:** While the 8th thread still provides a performance boost, the efficiency is lower because logical threads share execution resources within a single physical core.

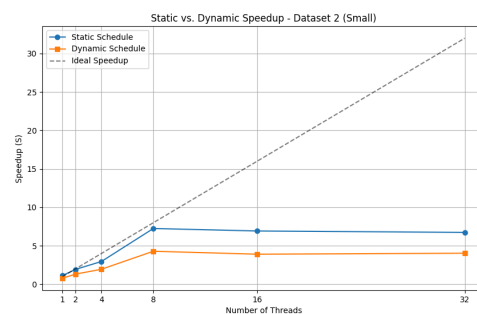
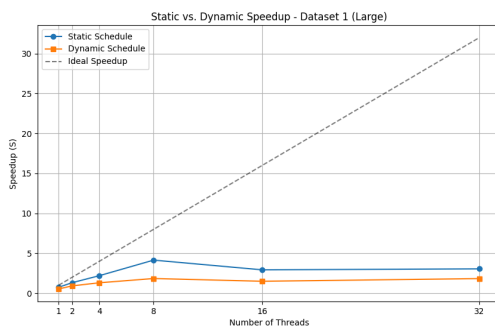


Plot2: Efficiency vs threads (Dataset 1 and Dataset 2)

5.4. Effect of Scheduling Parameters

We compared Static and Dynamic scheduling across all thread counts, you can see the comparison on both datasets on the *plots 3 & 4*:

- **Static Scheduling:** Consistently provided the best results. Since our 9x9 kernel performs a fixed number of operations per pixel, the workload is naturally balanced, making the low-overhead nature of static scheduling ideal.
- **Dynamic Scheduling:** Showed significantly lower performance, particularly in Dataset 2. This is due to the high synchronization overhead required for threads to request new work chunks, which outweighs the benefits for this type of uniform image processing.



Plot3 & 4: Static vs dynamic speedup (Dataset 1 and Dataset 2)

5.5. Bottleneck Analysis: What limits scalability?

Two main factors limit the scalability of our implementation:

1. **Sequential Phase Dependency:** The Opening operation is inherently divided into two distinct stages: Erosion followed by Dilation. In our implementation, there is a mandatory synchronization barrier between these phases. No matter how fast a thread finishes its portion of the Erosion, it must wait for all other threads to finish before it can start the Dilation. This "Stop-and-Wait" behavior limits the overall fluid scaling of the program.
2. **Hardware Core Limit:** The Intel i7-8550U has only 4 physical cores. Beyond 4 threads, performance gains rely on Hyper-Threading, and beyond 8 threads (the logical limit), the system suffers from extreme context-switching overhead, causing execution times to increase again. Also in almost all the occasions, the efficiency decreases when we try to use more than 4 threads.
3. **Memory Bandwidth and Cache Saturation:** Morphological operations are highly **memory-bound**. For every pixel, the CPU must read 81 neighbors (9x9 kernel) but performs very simple calculations.
 - **Dataset 1 (Large images):** the high volume of data exceeds the CPU's cache capacity, forcing the cores to wait for data from the RAM. This "Memory Wall" limits the speedup despite the large workload.
 - **Dataset 2 (Small images):** the data fits more efficiently within the L3 Cache, which explains why it achieved a much higher speedup (7.25x) compared to the larger dataset.

5.6. Visual results

Figure 1 and *Figure 2* show the outputs of the execution, as you can see in the *figure1* is easier to check the affected parts (the image is more clear, clean the imperfections) and in the *figure 2* you can see how the water body is easier to detect using any type of visual computation algorithm.

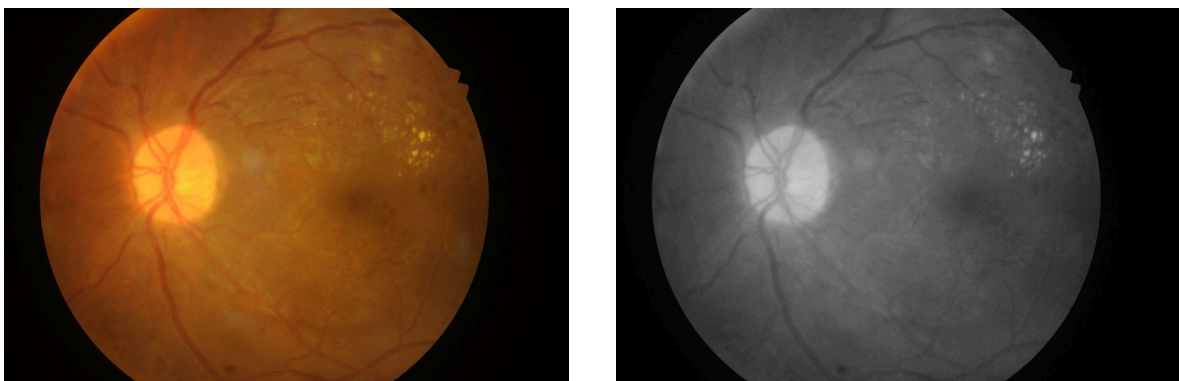


FIGURE 1: images from indian diabetic retinopathy image dataset and results after morphology

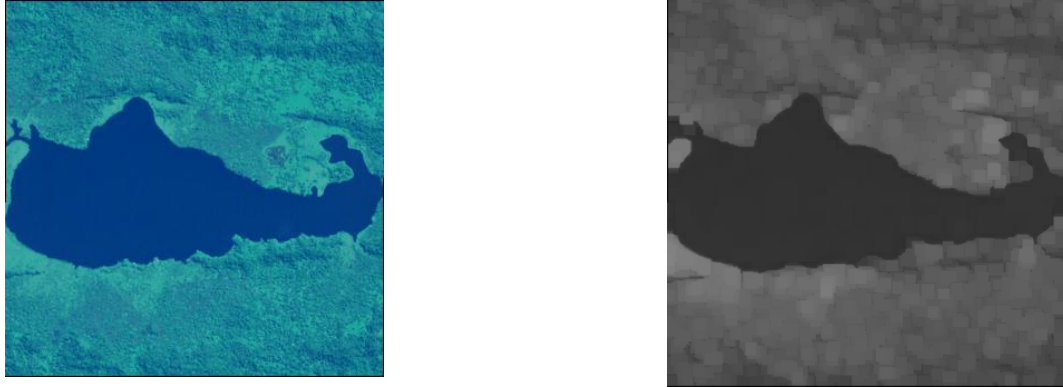


FIGURE 2: images from Satellite Images of Water Bodies Dataset and results after morphology

6. Discussion

6.1. Interpretation of Results

Plot 1: Speedup vs. Threads (Dataset 1 and 2)

Plot 1 shows that speedup peaks at 8 threads for both datasets, matching the hardware's 8 maximum processors (with hyper threading). Dataset 2 (Water) achieves a significantly higher maximum speedup (7.25x) than Dataset 1 (4.14x). This gap is caused by the "Memory Wall": the high-resolution images in Dataset 1 saturate the RAM bandwidth, while the smaller images in Dataset 2 benefit from staying within the faster CPU cache.

Plot 2: Efficiency vs. Threads (Dataset 1 and 2)

Plot 2 highlights a super speedup for Dataset 2 with an efficiency of 1.11 at 1 thread, suggesting the parallel code handles memory more efficiently than the sequential baseline. Efficiency remains high up to 4 threads but drops at 8 threads as Hyper-Threading begins sharing physical core resources. Beyond 8 threads, efficiency collapses due to extreme context-switching overhead as 32 threads compete for only 8 logical cores.

Plots 3 & 4: Static vs. Dynamic Speedup

These plots confirm that **Static scheduling** consistently outperforms Dynamic scheduling across all thread counts. Since the 9x9 kernel performs a constant number of operations per pixel, the workload is perfectly balanced by nature. Dynamic scheduling introduces an unnecessary management tax because threads must constantly synchronize to request new work, whereas Static scheduling assigns work with near-zero overhead.

6.2. Interpretation of Results and Theoretical Expectations

The experimental results demonstrate a significant performance improvement through parallelization, though they deviate from ideal linear scaling due to hardware and algorithmic constraints.

- **Amdahl's Law:** Our results confirm that the speedup is limited by the sequential fraction of the code, such as image I/O and OpenMP environment initialization. This is clearly visible in *plot 1*, where neither dataset reaches the "Ideal Speedup" line.
- **Gustafson's Law:** Interestingly, the smaller Dataset 2 scaled better than the larger Dataset 1. While Gustafson's Law suggests larger workloads scale better, our implementation hit a "Memory Wall." The high-volume workload of Dataset 1 exceeded the CPU cache capacity, leading to memory bandwidth saturation, whereas Dataset 2 fit efficiently within the cache.

6.3. Overhead and Bottleneck Analysis

- **Scheduling Overhead:** As shown in Plots 3 and 4, Static scheduling is much faster than Dynamic. Because every pixel takes the same time to process, the "management tax" of Dynamic scheduling, where threads constantly ask for more work, only slows the system down.
- **Hardware Limit:** The most significant bottleneck is the 4 physical cores of the i7-8550U. While 8 threads provide a boost via Hyper-Threading, efficiency drops after 4 threads because the physical resources are being shared.
- **The "16-32 Thread Crash":** Beyond the 8 logical threads, the CPU becomes oversubscribed. The Operating System spends more time swapping threads in and out than actually processing pixels, causing the performance to drop significantly.

6.4. Limitations and Constraints

- **Memory Bound:** The algorithm is limited by memory speed rather than CPU speed. Reading 81 pixels for every single output pixel creates a "Memory Wall" for large images.
- **Kernel Redundancy:** The current algorithm re-calculates the 9x9 window for every pixel, which is redundant. This limits how much speed we can gain regardless the number of threads.

6.5. Lessons Learned

- **Hardware awareness is key:** Knowing that we had 8 logical threads allowed us to find the "sweet spot" for performance. The logical (4 of them) cores give better efficiency than the ones that work with hyperthreading.
- **Dataset size matters:** Scaling behavior changes based on whether the data fits in the CPU cache or must be pulled from the slower RAM. Without following the normal operation that Gustafson's law tells us.
- **Simple is often better:** For uniform tasks like image filters, the simplest scheduling (Static) usually provides the best results.

6.6. Conclusions

In conclusion, parallelizing the morphological opening algorithm with OpenMP provides a significant performance boost, reducing execution times by up to 7.25x. The project successfully demonstrates that multi-threading is highly effective for pixel-independent image processing, provided the workload is balanced. However, performance is strictly

bounded by the 8-thread hardware limit and memory bandwidth bottlenecks, which prevent ideal linear scaling in larger datasets. Ultimately, this study confirms that for uniform kernels, Static scheduling is the most efficient approach to maximize resource utilization and minimize synchronization overhead.

6.7. Summary of achievements

- Developed a multi-threaded Morphological Opening algorithm in C++ using OpenMP, achieving a peak speedup of 7.25x.
- Validated the parallel implementation against a sequential to ensure that the parallel code works as well as the sequential.
- Comparing the static scheduling vs dynamic scheduling.
- Analyze the results, understand them and get to a conclusion.

6.8. Best configuration found

The optimal configuration for the i7-8550U is 8 threads using Static scheduling. This maximizes logical core utility while avoiding the synchronization overhead of dynamic task management. Peak efficiency is achieved with medium-resolution images that fit within the L3 cache, preventing memory bandwidth saturation.

6.9. Future work suggestions

- **Hardware acceleration:** trying the code in a computer with more cores and L3 cache capacity.

7. References

[1] AaryaPatel98. **Indian Diabetic Retinopathy Image Dataset**. Kaggle Dataset Repository URL.

<https://www.kaggle.com/datasets/aaryapatel98/indian-diabetic-retinopathy-image-dataset/code/discussion>

[2] Francisco Escobar. **Satellite Images of Water Bodies**. Kaggle Dataset Repository URL. <https://www.kaggle.com/datasets/franciscoescobar/satellite-images-of-water-bodies/discussion?sort=hotness>

[3] **Parallel Programming Course Lecture Notes and Slides**. Course material from the Parallel Programming course, [Semester/Year].